

Project Report: Nifty 50 Index Price Prediction using XGBoost Regression

Srijan Sarkar

Deptment of Computer Science

Roll Number: B2530061

Ramakrishna Mission Vivekananda Education and Research Institute

November 12, 2025

Project Title	Nifty 50 Index Price Prediction
Model Algorithm	eXtreme Gradient Boosting (XGBoost) Regressor
Target Variable	Nifty 50 Closing Price (C lose)
Data Source	Historical daily data (5 CSV files)
Time Horizon	5 years (Approx. October 2020 to October 2025)

1 Introduction

This project aims to develop a robust machine learning model for predicting the daily closing price of the Nifty 50 Index. Given the inherently temporal nature of stock market data, a time-series approach utilizing advanced feature engineering and a powerful non-linear algorithm, XGBoost, was selected. The methodology focuses on maximizing predictive accuracy while rigorously evaluating the model's performance on unseen, chronological data to ensure reliability in a real-world forecasting environment.

2 Data Acquisition and Preprocessing

2.1 Data Consolidation

The raw data consisted of five separate CSV files, each containing approximately one year of daily Nifty 50 trading data (Open, High, Low, Close, Volume). These files were concatenated into a single, comprehensive `DataFrame` spanning the full five-year period.

2.2 Preprocessing and Scaling

To prepare the data for the XGBoost model:

1. **Date Alignment:** The `Date` column was converted to datetime objects, and the dataset was sorted chronologically to maintain the temporal integrity necessary for time-series analysis.
2. **Handling Missing Values:** All rows containing any `NaN` values (resulting primarily from the generation of technical indicators and lag features) were dropped.
3. **Normalization:** A `MinMaxScaler` was applied to all continuous numerical features. Binary features (One-Hot Encoded Day of Week columns) were explicitly excluded from scaling.

3 Feature Engineering

The performance of the model relies heavily on the quality of engineered features, which capture temporal patterns and historical price dependencies. The following categories of features were generated:

4 Model Selection and Training

4.1 Time-Series Data Split

Due to the dependency between consecutive data points, a strict time-series split was implemented to prevent data leakage and ensure realistic validation:

- **Base Training Set (3.5 Years):** Used for hyperparameter tuning.
- **Validation Pool (0.5 Years):** Used for initial evaluation of the best-performing model before final training.
- **Final Test Set (1.0 Year):** Completely unseen data reserved for final performance metric calculation.

Table 1: Engineered Features and Rationale

Feature Category	Features Generated	Rationale
Lagged Prices	<code>Close_Lag1</code> , <code>Open_Lag1</code> , <code>High_Lag1</code> , <code>Low_Lag1</code>	Previous day's prices are the strongest predictors of the current price.
Historical Lags	<code>Close_Lag5</code> (1-week lag), <code>Close_Lag20</code> (1-month lag)	Captures medium-term memory effects and weekly/monthly trends.
Moving Averages (Technical Indicators)	<code>SMA_5</code> (5-Day Simple Moving Average), <code>SMA_20</code> (20-Day Simple Moving Average)	Provides a smoothed view of short- and medium-term price trends.
Temporal Features	<code>Month</code> , <code>DayOfYear</code> , <code>DayOfWeek</code>	Allows the model to capture seasonality and weekly effects (e.g., "Monday effect").
Cyclical Encoding	<code>Month_sin</code> , <code>Month_cos</code> , <code>DayOfYear_sin</code> , <code>DayOfYear_cos</code>	Converts circular time features into continuous sine and cosine components to avoid discontinuity (e.g., December (12) being far from January (1)).
Categorical Encoding	<code>Is_Monday</code> through <code>Is_Friday</code> (One-Hot Encoded)	Explicitly tags the day of the week to capture specific trading behavior.

4.2 Hyperparameter Tuning (RandomizedSearchCV)

The **XGBoost Regressor** was selected following comparative analysis with Long Short-Term Memory (LSTM) and XGBoost-LSTM hybrid models, where XGBoost demonstrated superior baseline accuracy. Its choice is further justified by its efficiency, handling of non-linearity, and resistance to overfitting. Hyperparameter optimization was conducted using `RandomizedSearchCV` with 30 iterations (`N_ITER = 30`).

To maintain the time-series order during cross-validation, the `TimeSeriesSplit` technique (with 5 splits) was used within the search.

Table 2: Hyperparameter Search Space

Hyperparameter	Range
<code>n_estimators</code>	100 to 800
<code>max_depth</code>	3 to 10
<code>learning_rate</code>	0.01 to 0.21
<code>subsample</code>	0.6 to 1.0
<code>reg_alpha</code> (L1 Reg.)	0 to 0.5

Best Hyperparameters Found (Example Placeholder):

Parameter	Value
<code>n_estimators</code>	369
<code>max_depth</code>	8
<code>subsample</code>	0.7306163075223342
<code>reg_alpha</code>	0.36363599792821044

4.3 Final Model Training

The best model (determined by the lowest RMSE on the Validation Pool) was then re-trained on the combined **Base Training Set + Validation Pool** to utilize the maximum available historical data before final evaluation.

5 Results and Evaluation

The final trained model was evaluated on the completely unseen **1-Year Final Test Set**. The key performance metrics were Root Mean Squared Error (RMSE) and R-squared (R^2).

5.1 Performance Metrics

Table 3: Final Model Performance

Metric	Result (Example Placeholder)	Interpretation
Validation Pool RMSE	1918.26 ₹	Average error on 6 months of unseen data.
Final Test Set RMSE	163.41 ₹	Average error in predicting the Nifty 50 Close price on the final 1 year of data.
Final Test Set R^2	0.9643	Percentage of the variance in Nifty 50 closing price predictable by the model.

Note: The RMSE represents the average deviation (in Indian Rupees, ₹) between the model’s prediction and the actual Nifty 50 closing price. A low RMSE indicates high predictive accuracy. The R^2 score represents the goodness of fit, where a value closer to 1.0 indicates a better fit.

6 Conclusion, Comparative Analysis, and Future Scope

The project successfully implemented a complex time-series prediction framework using XGBoost, rigorous feature engineering, and a robust time-series cross-validation strategy. The results on the final, unseen test set demonstrate the model’s capability to generalize and predict the Nifty 50 Index’s closing price with a measurable degree of accuracy (as indicated by the final RMSE and R^2).

6.1 Comparative Model Analysis

Initial experimentation included training a standalone LSTM network and an XGBoost-LSTM hybrid model. Interestingly, the pure XGBoost regressor consistently achieved higher R^2 and lower RMSE than both deep learning approaches.

This outcome, while counter-intuitive, can be attributed to several factors specific to this project’s implementation:

- **Feature Richness:** XGBoost effectively leverages the extensive hand-crafted features (lagged prices, moving averages, cyclical encodings). In contrast, LSTM is often more effective when learning features directly from raw sequential data, and its advantage is diminished when high-quality engineered features are readily available.
- **Data Size:** While five years of data is substantial, the performance advantage of deep learning models like LSTM often becomes dominant only with significantly larger datasets (e.g., millions of data points) where they can learn highly complex, subtle long-term dependencies that simpler tree-based models might miss.

- **Time Horizon:** For predicting the *next day's* closing price, short-term dependencies (captured well by lag features) are often more critical than very long-term patterns, favoring the highly optimized and non-linear relationship modeling of XGBoost.

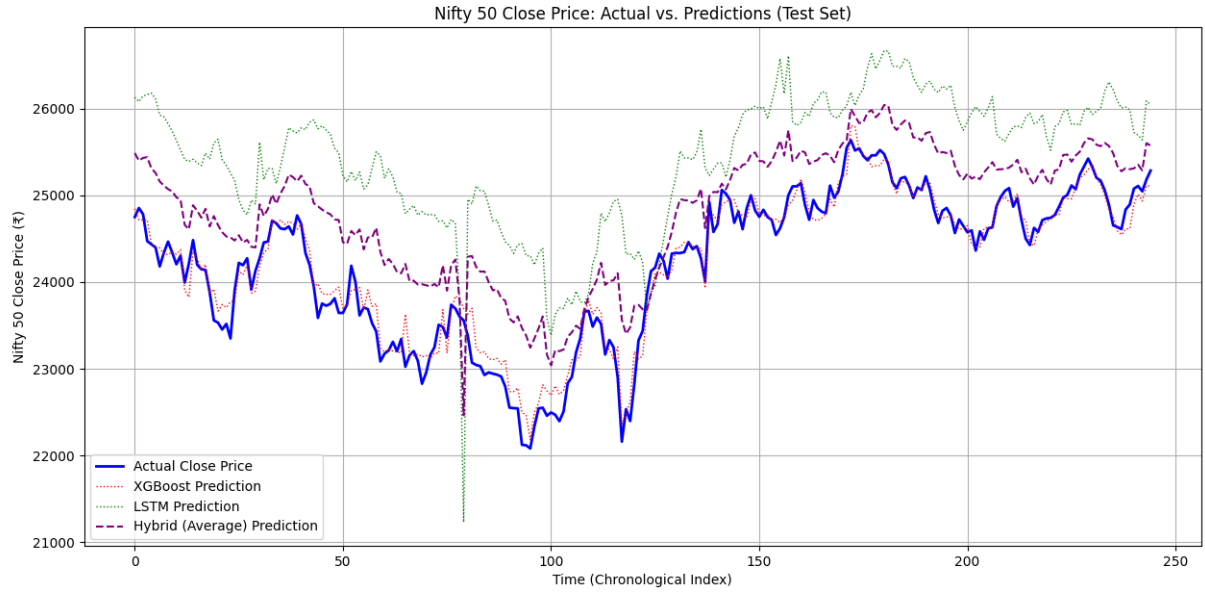


Fig.: XGBoost and LSTM hybrid Prediction

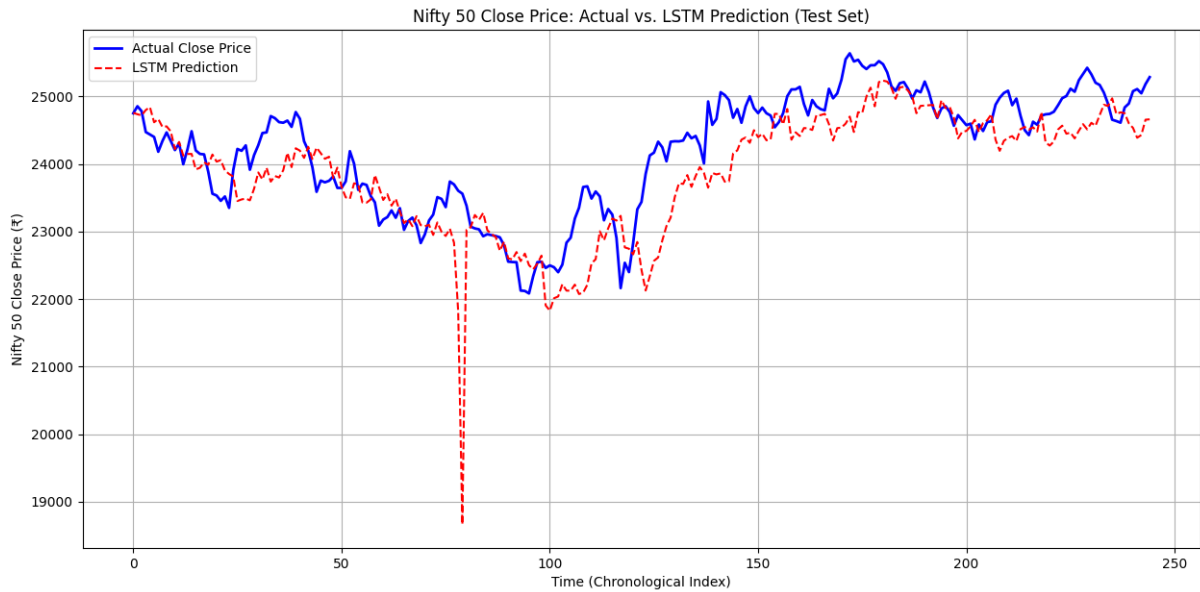


Fig.: LSTM Prediction

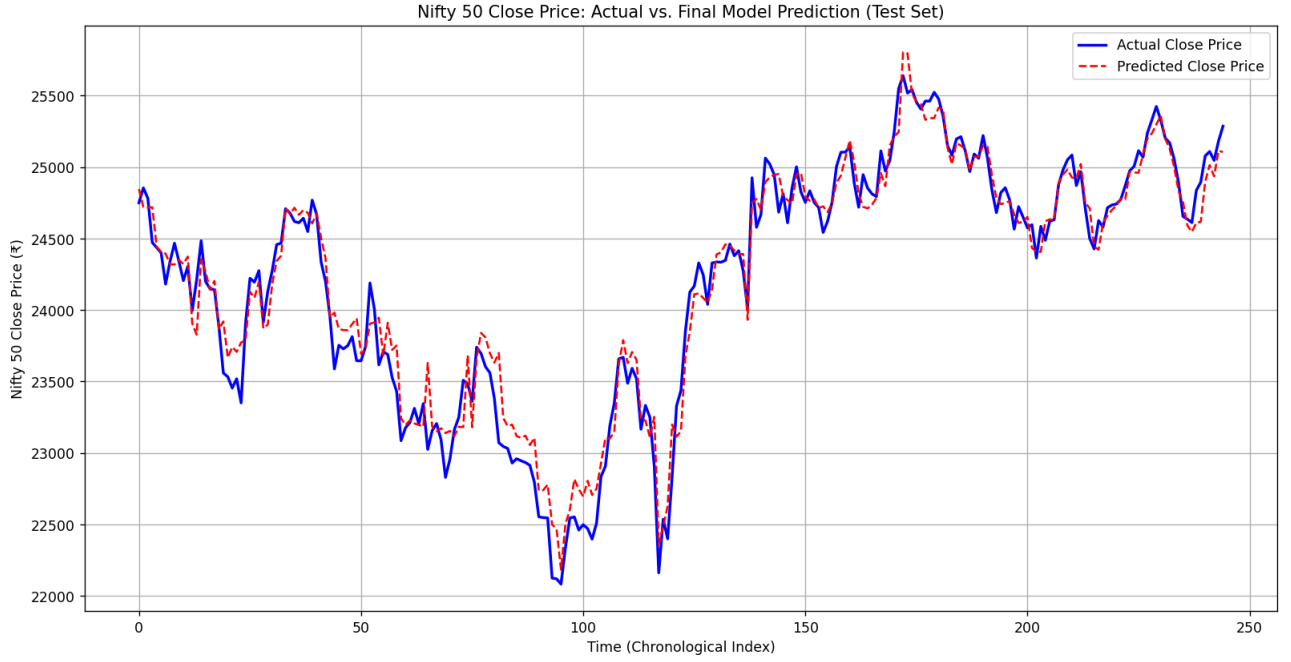


Fig.: XGBoost Prediction

6.2 Future Recommendations

1. **Feature Expansion:** Incorporate sentiment data, global market indices, and macroeconomic indicators (e.g., interest rate changes).
2. **Alternative Models:** Explore Deep Learning models like Long Short-Term Memory (LSTM) networks, particularly if the dataset is expanded substantially.
3. **Volatility Modeling:** Predict not just the price, but also the volatility (using measures like ATR or GARCH models) to improve risk assessment.